

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

Nombre: Angela Jimena Santizo Escobar

Carnet: 1207925

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH

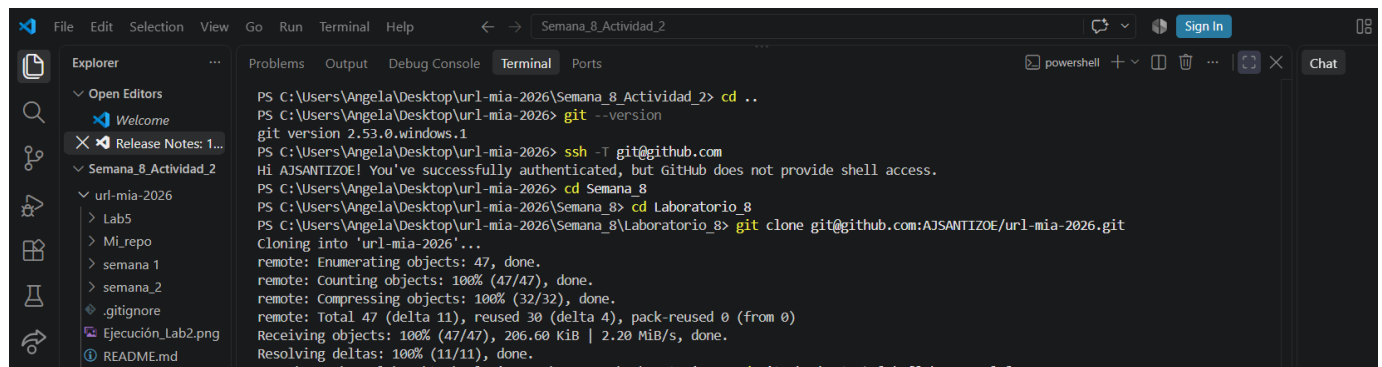
Verifica Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

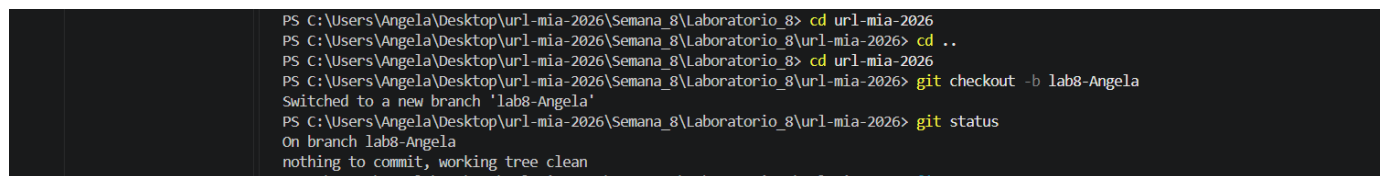
Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo.

Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_.git`

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`



```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8_Actividad_2> cd ..
PS C:\Users\Angela\Desktop\url-mia-2026> git --version
git version 2.53.0.windows.1
PS C:\Users\Angela\Desktop\url-mia-2026> ssh -T git@github.com
Hi AJSANTIZOE! You've successfully authenticated, but GitHub does not provide shell access.
PS C:\Users\Angela\Desktop\url-mia-2026> cd Semana_8
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8> cd Laboratorio_8
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8> git clone git@github.com:AJSANTIZOE/url-mia-2026.git
Cloning into 'url-mia-2026'...
remote: Enumerating objects: 47, done.
remote: Counting objects: 100% (47/47), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 47 (delta 11), reused 30 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (47/47), 206.60 KiB | 2.20 MiB/s, done.
Resolving deltas: 100% (11/11), done.
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8> cd ..
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8> cd url-mia-2026
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\url-mia-2026> git checkout -b lab8-Angela
Switched to a new branch 'lab8-Angela'
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\url-mia-2026> git status
On branch lab8-Angela
nothing to commit, working tree clean
```



```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8> cd url-mia-2026
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> cd ..
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8> cd url-mia-2026
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> git checkout -b lab8-Angela
Switched to a new branch 'lab8-Angela'
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\url-mia-2026> git status
On branch lab8-Angela
nothing to commit, working tree clean
```

2. Crear un archivo personalizado

Crea un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: mensaje_LuisRojas.json

Contenido:

```
{
  "nombre": "TU NOMBRE",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "[tu_carne]",
  "archivo": "mensaje_[Nombre_apellido].txt"
}
```

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> @'
>> {
>>   "nombre": "Angela Santizo",
>>   "curso": "Manejo e Implementación de Archivos",
>>   "carne": "1207925",
>>   "archivo": "mensaje_AngelaSantizo.txt"
>> }
>> '@ | Set-Content -Encoding UTF8 mensaje_AngelaSantizo.json
```

3. Cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión **.enc**

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact`

Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

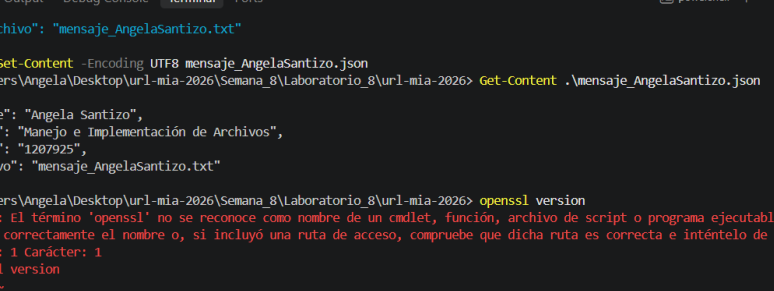
Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"

3.3 Cifra tu archivo

Comprueba dónde quedó instalado: `Get-Childitem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`
`[Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")`

Actualiza el PATH de esta terminal inmediatamente: `$env:Path += ";C:\Program Files\OpenSSL-Win64\bin"`



The screenshot shows a Windows PowerShell terminal window with the following content:

```

Go Run Terminal Help
← → Semana 8_Actividad_2

Problems Output Debug Console Terminal Ports
powershell + - [ ] ... [ ]

>> "archivo": "mensaje_AngelaSantizo.txt"
>> }
>> @ | Set-Content -Encoding UTF8 mensaje_AngelaSantizo.json
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> Get-Content .\mensaje_AngelaSantizo.json
{
  "nombre": "Angela Santizo",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "1207925",
  "archivo": "mensaje_AngelaSantizo.txt"
}
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> openssl version
openssl : El término 'openssl' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable. Compruebe si
escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ openssl version
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (openssl:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> winget install --id ShiningLight.OpenSSL.Light --exact
Encontrado OpenSSL.Light [ShiningLight.OpenSSL.Light] Versión 4.0.2
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Este paquete requiere las siguientes dependencias:
- Paquetes
  Microsoft.VCRedist.2015+.x64
Descargando https://slproweb.com/download/win64OpenSSL_Light-4_0_2.msi
5.73 MB / 5.73 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026>
  
```

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_AngelaSantizo.json -out .\mensaje_AngelaSantizo.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8> $opensslPath = "C:\Program Files\OpenSSL-win64\bin"
>> [Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")
>> $env:Path += ";C:\Program Files\OpenSSL-win64\bin"
>> openssl version
OpenSSL 4.0.2 25 Aug 2026 (Library: OpenSSL 4.0.2 25 Aug 2026)
```

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_AngelaSantizo.json -out .\mensaje_AngelaSantizo.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> Get-ChildItem

Directorio: C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026

Mode                LastWriteTime         Length Name
----                -
d-----          28/08/2026         08:54         Lab5
d-----          28/08/2026         08:38         Mi_repo
d-----          28/08/2026         08:38         semana 1
d-----          28/08/2026         08:38         semana 2
-a-----          28/08/2026         08:38             15 .gitignore
-a-----          28/08/2026         08:38        159646 Ejecución_Lab2.png
-a-----          28/08/2026         09:00         176 mensaje_AngelaSantizo.enc
-a-----          28/08/2026         08:46        152 mensaje_AngelaSantizo.json
-a-----          28/08/2026         08:38         16 README.md
-a-----          28/08/2026         08:38        200 README.md.txt
-a-----          28/08/2026         08:38           2 Semana_7
```

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc`

¿Qué observaste?

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> Get-Content .\mensaje_AngelaSantizo.enc
Salted_ñ 1A.B5_°=æ$mm/ÖÊô l"¥Öh2²(2ÖÖk-,†çéuÄ{µC$Ê7&v4*NvEúP"Ööİ
\Ö'Ü-è,ä%yEFÊ$DÊ$V!6%JCüXi! @t,¼<~^YÖ- :?@l.r|Aü7±8±êIpbŸ2ÊBÄ¹±çeZ0ß+Dİ]Ê=|®BVyb-ääâaQ\İ&{]
```

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

`openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out .\mensaje_TuNombre_recuperado.json`

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_AngelaSantizo.enc -out .\mensaje_AngelaSantizo_recuperado.json
enter AES-256-CBC decryption password:

PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> Get-Content .\mensaje_AngelaSantizo_recuperado.json
{
  "nombre": "Angela Santizo",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "1207925",
  "archivo": "mensaje_AngelaSantizo.txt"
}
```

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json`

¿Cuál es el HMAC-SHA256 obtenido?

```
PS C:\Users\Angela\Desktop\url-mia-2026\Semana_8\Laboratorio_8\url-mia-2026> openssl dgst -sha256 -hmac "miclaveangela2026" .\mensaje_AngelaSantizo.json
HMAC-SHA2-256(.\mensaje_AngelaSantizo.json)= bafac11a8d40469624dc305aecdc70a6b2cfb195062ca7a8423df719e4d987a0b
```

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?

- Confidencialidad, sin la contraseña, el archivo permanece ilegible, aunque el atacante tenga acceso al archivo cifrado.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

- Porque el repositorio queda accesible a otras personas (y su historial se conserva), así que si la contraseña o la clave secreta quedan ahí, cualquiera puede descifrar el archivo o falsificar la huella, anulando toda la protección.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

- SHA-256 es un hash simple que cualquiera puede calcular; solo garantiza integridad. HMAC-SHA256 combina el hash con una clave secreta, así que además garantiza autenticidad y solo quien conoce la clave puede generar o verificar la huella correctamente.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.